Master Thesis

NL2Vis: Multi-agent System for Natural Language to Data Visualization

Jacob Chen
(matriculation number 1980440)

May 4, 2026

Submitted to
Social Data Science Group
Prof. Dr. Marc Ratkovic
University of Mannheim

Abstract

This thesis investigates the development of autonomous, multi-agent systems for the Natural Language to Visualization (NL2Vis) task, focusing on overcoming the limitations of existing specialized frameworks. Current state-of-the-art systems tend to excel in one of two key areas: sophisticated, multi-table database querying or high-fidelity, iterative visual refinement, but rarely both. Furthermore, these systems almost exclusively rely on costly, proprietary Large Language Models (LLMs). This work proposes *LocalVisAgent*, a novel hybrid multi-agent framework that integrates the robust database-processing architecture of `nvAgent` with a multimodal visual feedback loop inspired by `MatPlotAgent`. This entire framework is implemented using locally-run, open-source LLMs, including Llama 3 and LLaVA, to ensure cost-effectiveness and data privacy. We evaluate this hybrid agent against two distinct, specialized benchmarks: `VisEval`, to test complex database interaction, and `MatPlotBench`, to test visual and aesthetic accuracy. Our results demonstrate that `LocalVisAgent` successfully retains the strong database querying performance of its baseline while dramatically improving visual fidelity on complex plotting tasks. This work validates the efficacy of a hybrid, local-first approach, offering a viable and comprehensive solution that bridges the gap between database-centric and visualization-centric NL2Vis systems.

Contents

Abstract **ii**

1. Introduction **1**
1.1. Research Motivation . . . 1
1.2. Approach and Scope . . . 1
1.3. Contributions . . . 2

2. Literature Review **3**
2.1. The NL2Vis Pipeline and What Makes a Good Visualization . . . 3
2.2. From Rule-Based Systems to Large Language Models . . . 4
2.3. Multi-Agent LLM Frameworks for NL2Vis . . . 5
2.4. Open Challenges in Deploying NL2Vis Systems . . . 6
2.4.1. Ambiguity and Intent Underspecification . . . 6
2.4.2. Scalability and Context Constraints . . . 6
2.4.3. Privacy, Data Confidentiality, and the Cost of Proprietary APIs . 7
2.4.4. Evaluation and Human-in-the-Loop Feedback . . . 7
2.5. Open-Source LLMs and Local Deployment . . . 7
2.6. Evaluation Benchmarks: From nvBench to VisEval . . . 8
2.7. Summary and Research Gap . . . 9

3. Methodology **11**
3.1. Objective and Scope Assumptions . . . 11
3.2. Pipeline Architecture . . . 11
3.2.1. Processor . . . 12
3.2.2. Composer . . . 12
3.2.3. Validator . . . 12
3.2.4. Methodological Scope and Known Tensions . . . 13
3.3. Prompting and Refinement Protocol . . . 13
3.4. Local Inference Stack . . . 13
3.5. Reproducibility by Design . . . 14

4. Experimental Setup **15**
4.1. Research Questions . . . 15
4.2. Benchmark and Scope . . . 15
4.3. Model Inventory and Serving Configuration . . . 15
4.4. Metrics and Pass Criteria . . . 16
4.5. Experimental Protocol . . . 16
4.6. Exploratory Reviewer-Agent Study . . . 17

4.7. Reliability and Reporting Constraints 17

5. Results and Evaluation **18**
5.1. Main Performance on VisEval After Dataset Update 18
5.2. Failure Composition: Invalid vs. Illegal 19
5.3. Reviewer-Agent and Ablation Diagnostics 19
5.4. Discussion 19

6. Conclusion **21**
6.1. Limitations 21
6.2. Future Work 22

Bibliography **23**

A. Additional Experimental Results **26**
A.1. Inference Performance Analysis 26
A.2. Ablation Study Results 26
A.3. Check Failure Analysis 26
A.4. Quantization Impact Comparison 26

B. Ehrenwörtliche Erklärung **32**

1. Introduction

The ability to extract insights from data is a central driver of progress in science, policy, and industry. In practice, however, access to that insight is often limited by technical barriers: many domain experts can interpret results, but cannot easily produce them from raw relational data (Wu et al. 2024). Natural Language to Visualization (NL2Vis) aims to reduce this gap by allowing users to request visualizations in everyday language rather than through manual query writing or code authoring (Maddigan & Susnjak 2023).

An effective NL2Vis system must solve two tightly coupled sub-problems. First, it must perform robust **Text-to-SQL** translation, mapping user intent to correct queries over often complex, multi-table schemas (Zhu et al. 2024; Wu et al. 2024). Second, it must perform **Text-to-Code** generation, converting retrieved data into executable visualization code (for example in Python) that is both syntactically valid and semantically aligned with the user's request (Huynh & Lin 2025). Errors in either stage can invalidate the final chart.

Recent work has increasingly converged on LLM-based multi-agent systems for this pipeline (Li et al. 2024). A representative example is `nvAgent` (Ouyang et al. 2025), which decomposes the task into a Processor, Composer, and Validator and reports strong performance on `VisEval` (Chen et al. 2025). Yet this progress comes with a practical trade-off: leading systems still depend on proprietary models such as GPT-4o. For organizations handling sensitive data, this dependency is often infeasible.

1.1. Research Motivation

This thesis investigates a concrete practical question:

> *How far can locally-run, open-source LLMs take an `nvAgent`-style multi-agent NL2Vis system on realistic benchmark tasks?*

The motivation is twofold. First, privacy and governance constraints in enterprise and public-sector settings often prohibit sending schema context and query content to third-party APIs. Second, recent progress in open-source and code-specialized LLMs suggests that the performance gap to proprietary models may now be narrow enough to justify local deployment in many settings.

1.2. Approach and Scope

We use `nvAgent`'s architecture as the baseline and run a systematic model sweep with locally served open-source LLMs. The evaluated models range from 7B instruction-tuned

baselines (Qwen2.5-7B) to code-specialized variants (Qwen2.5-Coder 7B, 14B, and 32B) and a Mixture-of-Experts coder model (Qwen3-Coder-30B-A3B). All models are served through vLLM (Kwon et al. 2023) and quantized with AWQ where appropriate (Lin et al. 2024). Evaluation is conducted on `VisEval` (Chen et al. 2025), including both single-table and multi-table settings.

Beyond model scaling, we test two targeted optimizations: (i) a visual reviewer agent that critiques rendered plots, and (ii) prompt-level improvements to the Composer to strengthen alignment with the original user intent. The analysis shows that the reviewer does not improve pass rates in this setup; most failures are caused by incorrect data integration upstream, not by chart rendering quality.

1.3. Contributions

The main contributions of this thesis are:

1. A systematic empirical evaluation of open-source LLMs of different sizes and specializations within an `nvAgent`-style multi-agent NL2Vis pipeline.

2. A failure-mode analysis showing that data-integration errors, rather than code-generation errors, are the dominant bottleneck for smaller local models.

3. A practical deployment blueprint for privacy-preserving NL2Vis systems, including clear trade-offs between model capability, cost, and data-governance constraints.

2. Literature Review

Natural Language to Visualization (NL2Vis) sits at the intersection of natural language processing, data visualization, and human–computer interaction. At its core, the task asks a system to take a free-form query in everyday language and produce a chart or other visual representation from structured, relational data—without requiring the user to write code or formulate explicit queries (Wu et al. 2024). The appeal is clear: it democratizes data exploration, allowing business analysts, policymakers, and domain experts to interact with complex databases on their own terms. Yet the technical challenge is equally significant: the system must simultaneously resolve what the user means, retrieve the right data, and render an appropriate visualization—a pipeline that compounds errors across its stages.

This chapter reviews the methodological foundations and current state of the art in NL2Vis, with particular attention to the challenges and design decisions most relevant to this thesis. Our focus is on systems that use Large Language Models (LLMs) in multi-agent configurations, the practical constraints of deploying such systems with sensitive organizational data, and the benchmarks used to evaluate them. We structure the review around five themes: the evolution of NL2Vis methods, the emergence of multi-agent frameworks, the technical and practical challenges of real-world deployment, the viability of open-source local LLMs, and the benchmark landscape.

2.1. The NL2Vis Pipeline and What Makes a Good Visualization

The NL2Vis pipeline can be understood as the composition of two distinct sub-tasks. The first is **Text-to-SQL**: translating a natural language query into a SQL statement that retrieves the right data from a relational database (Zhu et al. 2024). This sub-task presents significant challenges of its own—the system must link tokens in the user's query to columns and tables in the schema, handle joins across multiple tables, and correctly infer aggregations and filters that the user has not made explicit. The second sub-task is **Text-to-Code**: taking the retrieved data and generating executable code—typically Python with Matplotlib or Plotly—that renders an appropriate chart (Huynh & Lin 2025). Errors at either stage compound: incorrect data retrieval produces a visually polished but semantically wrong chart, while faulty code generation fails execution entirely.

A good visualization is more than one that runs without error. It must faithfully reflect the user's analytical intent: the correct chart type, appropriate axis mappings, accurate aggregations, and readable labels (Maddigan & Susnjak 2023). Earlier work in

the field focused heavily on syntactic correctness—whether the output code compiles and renders—but the community has increasingly recognized that a chart can be syntactically valid and semantically wrong at the same time. A bar chart that displays aggregated sales by the wrong grouping dimension, or a line chart that inverts its temporal axis, is technically valid but analytically misleading. Defining and measuring visual quality rigorously remains an open problem, and evaluation methodology is itself an active area of research, as we discuss in Section 2.6.

2.2. From Rule-Based Systems to Large Language Models

Early NL2Vis research relied on hand-crafted grammars and semantic parsers. These systems would parse a query into structured slots—chart type, data columns, aggregation function—using carefully designed linguistic rules. They were interpretable and predictable within their designed scope, but brittleness under paraphrase and an inability to generalize beyond the rules' coverage severely limited their reach.

Neural sequence-to-sequence models changed the picture considerably. Treating NL2Vis as a machine translation problem—translating natural language into a visualization specification language—allowed systems like ncNet to learn mappings end-to-end from paired examples (Luo et al. 2022). ncNet introduced Transformer-based seq2seq models with visualization-aware training strategies, including attention forcing and chart-template conditioning, and demonstrated clear improvements over rule-based baselines. These models relied on the nvBench dataset (Luo et al. 2021), which provided the first large-scale paired corpus for the task and will be discussed in more detail in Section 2.6.

The rise of large language models shifted the paradigm again, and decisively. Rather than training specialized models on NL2Vis-specific data, recent work has exploited the strong code generation and instruction-following capabilities of LLMs through prompting and in-context learning. Chat2Vis demonstrated that prompting GPT-3 and ChatGPT could produce usable visualizations from natural language queries with minimal task-specific engineering (Maddigan & Susnjak 2023). Prompt4Vis extended this idea by combining example mining—retrieving semantically similar examples from a corpus—with schema filtering to reduce irrelevant context, achieving substantial gains through better in-context prompt construction (Li et al. 2025). LIDA proposed a grammar-agnostic pipeline driven entirely by LLMs, capable of producing visualizations in multiple output languages without being tied to any particular chart specification format (Dibia 2023). Together, these works established that modern LLMs, when appropriately prompted, can handle the reasoning demanded by NL2Vis without task-specific fine-tuning—at least in the single-table, clean-data regime.

The methodological shift toward LLMs has been substantial. The emergence of accessible LLM APIs in 2022 and 2023 catalyzed a rapid expansion of LLM-based approaches to NL2Vis, with systems increasingly relying on prompting and in-context learning rather than task-specific model training, a trend documented across multiple recent empirical studies (Wu et al. 2024; Maddigan & Susnjak 2023; Li et al. 2025).

2.3. Multi-Agent LLM Frameworks for NL2Vis

A single LLM handling the entire NL2Vis pipeline faces a fundamental tension: the skills required to process a complex multi-table SQL query are not the same as those needed to generate polished Matplotlib code, or to identify a visual error in a rendered chart. This observation has motivated a divide-and-conquer approach, where a multi-agent system assigns each stage of the pipeline to a specialized agent. LLM-based multi-agent frameworks have demonstrated strong performance in complex software engineering tasks generally (Li et al. 2024), and the NL2Vis community has adopted this paradigm quickly.

`nvAgent` (Ouyang et al. 2025) is the most directly relevant prior work for this thesis. It organizes the NL2Vis pipeline around three collaborating agents. The *Processor* agent handles the database interaction: given a user query and a database schema, it filters irrelevant tables and columns, classifies query complexity as single-table or multi-table, and generates an executable SQL query. This schema-filtering step is critical—it reduces the context size passed to downstream agents and helps the system focus on only the relevant portions of the schema. The *Composer* agent takes the Processor's output and generates a high-level visualization plan in a structured Visual Query Language (VQL) format, specifying chart type, axis mappings, and groupings before any code is written. The *Validator* agent translates this plan into executable Python code and runs it; if execution fails, the error message is fed back to the Composer for correction. Evaluated on the co-developed VisEval benchmark (Chen et al. 2025), nvAgent achieved pass rate improvements of 7.88% on single-table and 9.23% on multi-table queries over the best prior baseline, and maintained a roughly 20% margin over non-agentic approaches. The multi-table strength is particularly notable, as many earlier systems struggle when a visualization requires joining data across multiple tables.

Despite this strong database performance, nvAgent's Validator has an important limitation: it only detects code execution failures. A chart that runs successfully but displays the wrong data, uses an inappropriate chart type, or has missing labels is approved without question. This gap between syntactic success and semantic correctness is a central weakness, and one that motivates the visual feedback approaches in more recent systems.

`MatPlotAgent` (Yang et al. 2024) addressed this gap from the opposite direction, focusing on visual quality. Its architecture adds a multimodal feedback loop to the code generation pipeline: after a chart is rendered, a Vision Language Model (VLM) examines the image and provides natural language critique—noting, for example, that an axis label is missing or that the chart type does not match the request. This critique is fed back to the code generation module for iterative correction. MatPlotAgent demonstrated that even a small number of VLM-guided refinement iterations substantially improves visual fidelity on complex scientific visualization tasks.

`PlotGen` (Goswami et al. 2025) pushes this feedback idea further, decomposing the visual critique into three specialized agents: a Numeric Feedback Agent that verifies data values, a Lexical Feedback Agent that checks labels and titles, and a Visual Feedback Agent that assesses chart type and layout. This granular decomposition proved effective, yielding 4–6 percentage point improvements over strong baselines on complex visualization tasks. Similarly, Shi et al. trained a VLM specifically to judge visualization quality

and provide code-level repair suggestions, demonstrating that a model fine-tuned on examples of good and bad charts gives more actionable feedback than a general-purpose VLM used zero-shot (Shi et al. 2025).

These systems reveal a persistent pattern in the field: systems that excel at database querying (nvAgent) are weak at visual validation, and systems that produce high-quality visuals (MatPlotAgent, PlotGen) are typically demonstrated on pre-processed, single-file datasets that sidestep the database interaction entirely. No single framework has convincingly addressed both halves of the pipeline simultaneously. This *specialization gap* is one of the core architectural problems this thesis sets out to address.

2.4. Open Challenges in Deploying NL2Vis Systems

Beyond the specialization gap, the NL2Vis literature identifies a range of challenges that bear directly on the design choices made in this thesis. We discuss four that are most relevant here.

2.4.1. Ambiguity and Intent Underspecification

Natural language queries are rarely precise. Expressions such as "show me sales" or "compare regions" leave chart type, aggregation level, and time scope entirely unspecified. Systems must either make reasonable defaults or risk producing charts that are technically correct but analytically misleading (Maddigan & Susnjak 2023). Standard benchmarks deliberately use unambiguous prompts to facilitate reliable evaluation (Chen et al. 2025), which simplifies measurement but does not reflect the messiness of real-world use. This thesis inherits this scope assumption—working with well-specified queries—but acknowledges ambiguity handling as an important direction for future work. The development of benchmarks that include underspecified queries and multiple valid ground truths is an active area of discussion in the community.

2.4.2. Scalability and Context Constraints

Multi-agent NL2Vis systems consume substantial context window capacity. A real enterprise database may have dozens of tables and hundreds of columns; feeding the complete schema to an LLM in a single prompt is often impossible within the context limits of current models and quickly becomes expensive. Schema filtering, as implemented in nvAgent's Processor, is a practical response to this constraint (Ouyang et al. 2025). At the same time, iterative feedback loops add latency with each refinement round—a meaningful cost when running locally on hardware with limited parallelism. System design for NL2Vis thus requires balancing the benefits of iterative correction against the practical costs of inference time and context consumption.

2.4.3. Privacy, Data Confidentiality, and the Cost of Proprietary APIs

Perhaps the most significant practical barrier for real-world NL2Vis deployment is the tension between using state-of-the-art LLMs and protecting sensitive organizational data. The strongest NL2Vis systems rely on proprietary APIs—GPT-4o in the case of nvAgent (Ouyang et al. 2025), GPT-4V in MatPlotAgent (Yang et al. 2024)—which require sending the user's query and relevant portions of the database schema, and sometimes sample data rows, to third-party cloud servers. For organizations working with confidential or regulated data, this is frequently untenable.

Enterprise databases routinely contain information subject to strict data protection requirements. Public sector organizations—law enforcement agencies, healthcare providers, social services—handle data that may not legally or ethically be transmitted to external infrastructure. The same applies to corporate ERP systems, financial records, and customer databases. Even where external transmission is technically permitted, the financial cost scales with query volume and schema size, making API-based systems economically impractical at organizational scale. A further practical concern is reproducibility: proprietary APIs undergo silent model upgrades that can change output behavior without notice, making it difficult to audit or replicate results (Maddigan & Susnjak 2023). Running LLMs locally eliminates all of these concerns simultaneously—no data leaves the organization's infrastructure, inference is deterministic and auditable, and cost is bounded by hardware rather than per-token pricing (Lin et al. 2024).

2.4.4. Evaluation and Human-in-the-Loop Feedback

Current benchmarks assume a one-shot interaction model: a single query produces a single chart, which is scored against a ground truth. Real data exploration is rarely this clean. Users iterate—refining their query, rejecting an output and trying again, or asking follow-up questions. No widely adopted NL2Vis benchmark accommodates this kind of multi-turn interaction (Chen et al. 2025), and fully automating iterative human feedback for evaluation purposes remains an open problem. Within this thesis, we have explored whether an LLM can simulate the human feedback role to assess whether iterative correction can improve pass rates without requiring a human in the loop at each step.

2.5. Open-Source LLMs and Local Deployment

The argument for locally-run, open-source LLMs in NL2Vis is both practical and architectural. On the practical side, running LLMs locally eliminates data transmission to third parties, reduces per-query cost to amortized hardware cost, and ensures reproducibility across experimental runs (Lin et al. 2024). On the architectural side, smaller open-source models force a more careful allocation of roles within a multi-agent system, because no single small model can reliably handle the full pipeline in one shot.

The question of whether open-source models are capable enough for NL2Vis tasks has a nuanced answer. General-purpose instruction-tuned models in the 7–14B parameter

range perform meaningfully below GPT-4o on complex multi-table queries. However, domain-specialized variants change the picture substantially. Code-specialized models such as Qwen2.5-Coder are fine-tuned on large code corpora and show significantly stronger performance on code generation sub-tasks than their general-purpose counterparts of the same size (Wu et al. 2024). As model sizes increase toward 30–32B parameters, the gap between open-source coder models and proprietary APIs narrows to the point where the privacy and cost advantages begin to outweigh the residual performance difference for many practical use cases.

Efficient serving of local models is itself an engineering challenge. Loading and running a 14–32B parameter model requires careful memory management; naive inference implementations are too slow for interactive use. The vLLM serving framework addresses this through PagedAttention, an attention algorithm that borrows virtual memory and paging techniques from operating systems to achieve near-zero waste in key-value cache memory, enabling 2–4× higher throughput at the same latency compared to standard serving systems (Kwon et al. 2023). Weight quantization further reduces the memory footprint of large models. Activation-aware Weight Quantization (AWQ) identifies the small fraction of weights that contribute most to model outputs and protects them during quantization, achieving 4-bit compression with minimal accuracy loss (Lin et al. 2024). Together, vLLM and AWQ make it practically feasible to serve 14–32B parameter models on a single research GPU at throughput adequate for systematic benchmarking.

The role of model size in NL2Vis performance is not straightforward. Voigt et al. showed that small pre-trained language models can generate visualization specifications in real time on a CPU for simpler single-table tasks, establishing that reduced-size models can be viable for latency-sensitive deployments (Voigt et al. 2024). Their results also highlight that raw model size is not the only factor—prompt encoding, schema representation, and output grammar structure all substantially affect performance. This observation supports the multi-agent design philosophy: by giving each agent a narrow, well-defined sub-task, the system can use smaller models more effectively than a single large model handling everything at once.

2.6. Evaluation Benchmarks: From nvBench to VisEval

Standardized benchmarks are the backbone of empirical progress in NL2Vis. The dominant benchmark for most of the field's history has been **nvBench** (Luo et al. 2021), which contains over 25,750 paired natural language queries and Vega-Lite visualization specifications across 105 relational database domains and seven common chart types. nvBench was constructed by programmatically mapping SQL queries from the Spider text-to-SQL benchmark (Yu et al. 2018) into chart specifications, then generating corresponding natural language queries through a rule-based synthesis process. Its breadth made it the standard training and evaluation corpus across most of the systems discussed in this chapter (Luo et al. 2022; Li et al. 2025; Maddigan & Susnjak 2023).

nvBench has well-documented limitations, however. Its queries are synthetically generated and tend to be grammatically regular and fully specified—a poor match for the

noisy, underspecified queries typical of real users. Its evaluation is based on Vega-Lite specification matching, so systems trained on it may fail when asked to produce Python code output. Most importantly for this thesis, nvBench does not distinguish between single-table and multi-table scenarios in its standard evaluation, making it unsuitable for measuring performance on complex database queries involving joins.

VisEval (Chen et al. 2025) was designed to address these gaps in the LLM era. It provides 2,524 natural language and visualization pairs across 146 databases, with explicit coverage of both single-table and multi-table scenarios. Its evaluation framework goes substantially beyond simple string matching. A generated visualization is assessed along multiple dimensions: *validity* checks whether the code executes without error; *legality* verifies whether the retrieved data values in the chart match the ground-truth SQL output, catching charts that run correctly but display wrong information; and *readability* is assessed by an LLM judge evaluating label clarity, title appropriateness, and overall presentation quality. The primary metric, *pass rate*, requires a visualization to pass both the validity and legality checks. This tighter definition is significant: it catches exactly the category of errors that nvAgent's original Validator misses—charts that execute successfully but are semantically wrong because they integrate the data incorrectly.

VisEval was co-developed with nvAgent and has since become the standard benchmark for evaluating LLM-based multi-agent NL2Vis systems on database-querying tasks. It is the primary evaluation instrument for the experiments in this thesis. Its design constraints—unambiguous prompts, database ID provided to the system—narrow the scope relative to open-ended real-world deployment, but this precision is a strength for isolating and measuring specific capabilities in a controlled way.

2.7. Summary and Research Gap

The survey of the field reveals three converging gaps that this thesis is positioned to address. First, there is a *specialization gap*: the strongest database-querying system (nvAgent) lacks meaningful visual validation, while the strongest visual refinement systems (MatPlotAgent, PlotGen) sidestep database complexity. A system that handles both has not been demonstrated. Second, there is a *proprietary dependency gap*: every state-of-the-art NL2Vis system relies on closed-source models such as GPT-4o, and the performance of open-source, locally-run models in multi-agent NL2Vis configurations has not been systematically studied. Third, there is a *deployment gap*: the literature does not address the practical scenario of organizations that need NL2Vis capabilities for sensitive data that cannot leave their infrastructure.

This thesis takes nvAgent's architecture as its baseline and investigates how it performs when the proprietary LLM backbone is replaced with open-source models of varying size, specialization, and quantization level, all served locally via vLLM. We evaluate these configurations on VisEval, use the results to identify where smaller models fail, and explore targeted improvements to the agent design. The goal is not simply to match GPT-4o—that bar may require substantially larger models than those studied here—but

to characterize the frontier of what is achievable within the privacy and cost constraints that real-world organizational deployments impose, and to identify which components of the multi-agent pipeline are most responsible for performance differences.

3. Methodology

This thesis adopts an `nvAgent`-style multi-agent pipeline as the core method and evaluates how far locally served open-source models can support end-to-end NL2Vis on realistic database tasks (Ouyang et al. 2025). The methodological goal is practical rather than purely architectural novelty: preserve strong data integration behavior while operating under local deployment constraints (privacy, reproducibility, and bounded inference cost).

Method choices are grounded in prior literature before empirical validation. Specifically, the decomposition into specialized agents follows evidence that modular workflows improve robustness on complex NL2Vis and related multi-agent tasks (Ouyang et al. 2025; Li et al. 2024); structured schema reduction and prompt design are motivated by gains reported for context filtering and example-conditioned prompting (Li et al. 2025; Wu et al. 2024); and visual-feedback loops are treated as a testable extension rather than an assumed improvement, because reported benefits are benchmark- and task-dependent (Yang et al. 2024; Shi et al. 2025).

3.1. Objective and Scope Assumptions

The system is designed for single-turn NL2Vis generation under the same assumptions used by `VisEval`: prompts are intentionally unambiguous and the target database identifier is provided (Chen et al. 2025). This scope is deliberate. It isolates model and agent behavior on data retrieval and chart correctness before extending to multi-turn conversational interaction.

The primary evaluation target is therefore database-grounded correctness rather than open-ended aesthetic preference. Human-in-the-loop interaction and conversational memory are treated as future extensions, not as part of the core method validated in this thesis.

This controlled scope also reflects a known tension in the field. Real user requests are often ambiguous or underspecified (Maddigan & Susnjak 2023), and benchmark distributions may not perfectly match practical usage patterns (Nguyen et al. 2024). We therefore frame our method as a capability study under explicit assumptions, not as a claim that one-shot benchmark performance fully captures interactive production behavior.

3.2. Pipeline Architecture

The implemented pipeline follows a three-stage text workflow with optional visual review:

1. **Processor**: schema reduction and query-conditioned database reasoning,
2. **Composer**: structured visualization planning,
3. **Validator**: code generation, execution checking, and bounded refinement.

An optional reviewer-style visual loop was implemented as an exploratory extension; however, it is not part of the main method claim because the strongest gains in this study come from improving data integration quality upstream.

3.2.1. Processor

The Processor receives the user query and full database context, filters tables and columns, and produces an augmented schema representation for downstream agents. The practical purpose is to reduce prompt context while preserving columns required for correct joins, grouping, and filtering. This stage also emits a query complexity signal (single-table vs. multi-table) used for routing prompt templates.

The design contract of this stage is conservative recall over aggressive pruning: removing irrelevant schema context improves token efficiency, but omitting join-critical columns can induce legality failures downstream. This trade-off is consistent with prior observations that schema/context representation strongly affects NL2Vis and text-to-SQL outcomes (Wu et al. 2024; Li et al. 2025).

3.2.2. Composer

The Composer maps the processed schema context and user intent to a structured visualization query representation (VQL-like intermediate form). This representation explicitly captures chart type, selected fields, aggregation logic, grouping/order choices, and time binning when needed. By separating planning from final code rendering, this stage provides a stable interface for downstream validation and iterative repair.

The explicit intermediate representation is used to reduce error propagation from natural-language ambiguity into final code. In effect, the Composer acts as a semantic checkpoint: if intent grounding is weak here, later code refinement typically improves executability more than semantic correctness.

3.2.3. Validator

The Validator translates the intermediate representation into executable Python visualization code, executes it, and checks execution errors. If execution fails, the error trace is fed back into bounded refinement rounds. In the implemented setup, refinement is capped to a small fixed number of iterations to prevent unbounded latency and to keep comparisons across models fair.

This stage is intentionally execution-first but evaluation-aware: execution success is necessary yet insufficient, since `VisEval` legality checks reject semantically wrong but runnable charts (Chen et al. 2025). This distinction is central to the later failure analysis.

3.2.4. Methodological Scope and Known Tensions

Recent visual-feedback systems report meaningful gains through multimodal critique loops (Yang et al. 2024; Shi et al. 2025). At the same time, those gains are often demonstrated in settings where data retrieval is simplified relative to multi-table enterprise-style NL2Vis. Our methodology therefore treats visual critique as an optional diagnostic path and prioritizes data-grounding quality first. This is a conflict-aware design decision: it does not reject visual-feedback literature, but tests whether its benefits transfer under `VisEval`-style legality constraints and local-model limits.

3.3. Prompting and Refinement Protocol

Each agent uses role-specific prompts rather than one monolithic prompt. Processor prompts emphasize schema selection and relational grounding; Composer prompts emphasize representation correctness; Validator prompts emphasize executable code generation and repair. Decoding is deterministic ($T = 0$) to reduce run-to-run variability from sampling.

Refinement follows a strict stop policy:

1. stop on successful execution and valid structured output,
2. stop when maximum refinement rounds are reached,
3. mark remaining cases as failed for downstream evaluation.

This policy makes failure counts interpretable and reproducible across model variants.

We keep prompt workflow and stop conditions fixed across model variants to reduce confounding. In other words, observed differences are interpreted primarily as capability differences, not protocol differences. This choice follows standard benchmarking practice for controlled comparison, but it also limits exploration of adaptive prompting strategies that may improve absolute performance.

3.4. Local Inference Stack

All local model inference is served through vLLM (Kwon et al. 2023). We evaluate instruction-tuned and code-specialized Qwen-family models under constrained hardware using quantization where required (AWQ/FP8 configurations depending on model size) (Lin et al. 2024). This design reflects the deployment target of privacy-sensitive settings where external API calls are restricted.

The serving configuration keeps core generation parameters fixed across runs (context budget, output token cap, deterministic decoding), so observed differences are primarily attributable to model capability rather than aggressive hyperparameter retuning.

This stack is selected for deployment realism rather than maximal leaderboard performance. vLLM and quantization enable practical local serving of mid-to-large models, but quantization may alter behavior on difficult reasoning cases; accordingly, we interpret

efficiency gains and quality changes jointly instead of assuming monotonic improvements from either model scale or compression alone.

3.5. Reproducibility by Design

Methodologically, the pipeline is implemented with centralized configuration, structured logging of run outputs, and scriptable evaluation entry points. This supports consistent reruns across model variants and enables direct mapping from chapter claims to executable artifacts.

At the same time, we explicitly acknowledge a current reproducibility limitation: most comparisons are deterministic single-run evaluations rather than repeated stochastic trials. This is addressed in the experimental protocol through transparent reporting and, where feasible, additional robustness analysis.

Consequently, claims in this thesis are framed as directional and configuration-specific: they indicate where bottlenecks and improvements concentrate under fixed settings, while leaving full variance estimation and broader statistical generalization as future work.

4. Experimental Setup

This chapter defines the empirical protocol used to evaluate the local multi-agent pipeline under the scope above. The main text is intentionally centered on `VisEval` so that all reported outcomes are aligned with database-grounded NL2Vis correctness.

4.1. Research Questions

The experiments address three questions:

1. **RQ1:** How close can locally served Qwen-family models get to proprietary baselines on `VisEval` pass rate?
2. **RQ2:** Which failure categories (invalid vs. illegal) dominate as model size and specialization change?
3. **RQ3:** What performance-efficiency trade-offs emerge across quantized local model variants?

4.2. Benchmark and Scope

The primary benchmark is `VisEval` (Chen et al. 2025), which includes 2,524 natural-language/visualization pairs across 146 databases with both single-table and multi-table scenarios. We report results split by these two settings because multi-table performance is the dominant practical bottleneck for enterprise-style schemas.

No additional benchmark is required to support the core claims in this chapter. Exploratory visual-review experiments are reported separately and are not used to define the main contribution.

This benchmark choice prioritizes measurement reliability over interaction breadth. It offers strong legality-focused diagnostics, but does not cover iterative conversational correction or broad prompt ambiguity. We therefore treat benchmark conclusions as high-confidence for one-shot, database-grounded generation and lower-confidence for open-ended human-in-the-loop settings (Chen et al. 2025; Nguyen et al. 2024).

4.3. Model Inventory and Serving Configuration

Primary runs use locally served Qwen-family models, including instruction-tuned and code-specialized variants (7B to 32B class, plus MoE coder configuration), with quantization choices matched to available GPU memory.

For external reference points, we compare against reported `nvAgent` baselines using proprietary models from prior work (Ouyang et al. 2025). The emphasis of comparison is not strict parity of hidden infrastructure, but practical proximity under local constraints.

To keep comparisons fair:

- decoding temperature is fixed to deterministic generation,
- context and output budgets are held constant across local runs,
- prompt workflow and refinement limits are unchanged between model variants.

The experiment matrix follows four axes: (i) model specialization (general-instruction vs. coder-specialized), (ii) parameter scale, (iii) deployment format (quantized or non-quantized where available), and (iv) optional reviewer-agent activation. This structure directly supports RQ1–RQ3 by separating model capability effects from protocol effects.

Model selection is literature-informed rather than exhaustive. Coder-specialized models are prioritized because NL2Vis includes substantial code synthesis and repair demands (Wu et al. 2024); smaller models are retained as efficiency baselines (Voigt et al. 2024); and proprietary baselines are treated as reference ceilings from prior reports (Ouyang et al. 2025). Published evaluations also report strong dependence on visualization complexity, with larger drops on harder chart reasoning tasks, which further motivates explicit failure decomposition instead of single aggregate reporting (Khan et al. 2025).

4.4. Metrics and Pass Criteria

We follow `VisEval`'s metric decomposition (Chen et al. 2025):

- **Invalid rate:** failures from non-executable code or invalid output format,
- **Illegal rate:** executable outputs that fail semantic/data correctness checks,
- **Pass rate:** outputs that satisfy both validity and legality criteria.

When available, readability/quality scores from judge-based evaluation are reported as secondary diagnostics rather than primary success criteria, because the core bottleneck in our runs is data integration correctness.

This ordering reflects both evaluation logic and practical risk: an aesthetically acceptable chart with incorrect values is a higher-severity failure than a less polished chart with correct data grounding. The pass-rate decomposition is therefore the principal instrument for identifying where the pipeline fails.

4.5. Experimental Protocol

The main evaluation is a model sweep over local configurations with identical prompts and runtime settings. Each run produces per-query outcomes that are aggregated into single-table and multi-table summaries. We then compare trends across:

1. general instruction models vs. code-specialized models,
2. smaller vs. larger parameter scales,
3. unquantized vs. quantized deployment variants when both are available.

In addition to aggregate metrics, we inspect failure composition (invalid vs. illegal) to identify whether improvements come from better code execution stability or better semantic data grounding.

All runs use the same prompting workflow and stopping rules to preserve comparability. Where ablation or reviewer experiments are discussed, they are interpreted diagnostically: the purpose is to localize bottlenecks, not to claim universally superior architecture variants.

4.6. Exploratory Reviewer-Agent Study

A reviewer/visual-critic extension was tested as an exploratory add-on. In this thesis, it is treated as a diagnostic experiment rather than the main method: results indicate that gains are limited when upstream data correctness fails first. Consequently, reviewer-agent findings are reported as supporting evidence for bottleneck analysis, not as the primary source of pass-rate improvement.

This positioning intentionally contrasts with literature where visual-feedback loops improve outcome quality (Yang et al. 2024; Shi et al. 2025). Our interpretation is that transfer depends on failure regime: when legality errors dominate, additional visual critique may have lower marginal utility than upstream data-grounding improvements.

4.7. Reliability and Reporting Constraints

Where repeated trials are not computationally feasible, we report deterministic single-run outcomes with explicit caveats. This prevents overstating statistical certainty while preserving comparability across many model variants. The final interpretation therefore prioritizes consistent effect direction across splits and metrics over fragile point estimates.

To avoid post-hoc confirmation bias, result interpretation follows a fixed order: prior expectation from literature, observed behavior under the present setup, and then bounded conclusions tied to scope assumptions. Where conflicts with prior work arise, we report them explicitly and attribute them to plausible setup differences (benchmark design, bottleneck location, and local-resource constraints) rather than forcing artificial agreement.

5. Results and Evaluation

This chapter presents the main empirical findings of the local `VisEval`-based evaluation protocol introduced in Chapter 4. The analysis focuses on database-grounded correctness, reported through invalid rate, illegal rate, and pass rate, with results split into single-table and multi-table settings. We then examine where failures concentrate, discuss the reviewer-agent and ablation diagnostics, and conclude with the practical implications for privacy-preserving deployment.

5.1. Main Performance on VisEval After Dataset Update

Table 5.1 summarizes the strongest local runs after updating to the final `VisEval` release (full metric breakdown in Appendix A, Table A.1). Two trends stand out. First, coder-specialized models outperform similarly sized general instruction models, most clearly in multi-table queries. Second, scaling from 7B to 30B/32B primarily improves multi-table pass rate, which is the more challenging and practically relevant setting for enterprise-style schemas.

Relative to the initial 7B instruction baseline, the best multi-table run (Qwen3-Coder-30B-A3B-Instruct-FP8) improves pass rate from 37.25% to 70.32%, a gain of 33.07 percentage points. This indicates that practical local performance depends less on a simple "open-source versus proprietary" dichotomy and more on choosing the right model specialization and scale for the data-integration-heavy parts of the pipeline.

For context, the reported `nvAgent` proprietary-model baselines remain stronger at the top end (Ouyang et al. 2025). At the same time, the local results in this thesis narrow the practical gap while preserving privacy and reproducibility requirements that motivate local deployment.

Local Model	**Pass (Single)**	**Pass (Multi)**	**Illegal (Multi)**
Qwen2.5-7B-Instruct	62.64%	37.25%	32.98%
Qwen2.5-Coder-7B-Instruct-AWQ	65.08%	52.41%	34.58%
Qwen2.5-Coder-14B-Instruct-AWQ	73.54%	63.46%	26.73%
Qwen2.5-Coder-32B-Instruct-AWQ	75.67%	64.44%	26.13%
Qwen3-Coder-30B-A3B-Instruct-FP8	75.40%	70.32%	21.57%

Table 5.1.: Core local results on updated `VisEval`. Values are reported from the "After VisEval Update" block in Appendix A.

5.2. Failure Composition: Invalid vs. Illegal

Failure decomposition clarifies where further optimization is most needed. For smaller models, both invalid and illegal rates are high in multi-table cases. As model capability increases, invalid errors drop markedly, but illegal errors remain the dominant residual bottleneck. For example, moving from Qwen2.5-7B-Instruct to Qwen3-Coder-30B-A3B-Instruct-FP8 reduces multi-table invalid rate from 29.78% to 8.10%, while multi-table illegal rate remains 21.57%.

This pattern suggests that execution stability can be improved substantially with stronger coder models, whereas semantic data grounding (correct joins, grouping, and value mapping) remains harder. Put differently, the system increasingly "runs" before it is consistently "right." This interpretation aligns with the legality-oriented design of `VisEval`, which intentionally rejects executable but semantically incorrect charts (Chen et al. 2025).

Readability and quality are therefore treated as secondary diagnostics. Since readability checks are only evaluated after validity and legality pass, these scores are less informative for diagnosing the dominant failure source in lower-performing configurations.

5.3. Reviewer-Agent and Ablation Diagnostics

Reviewer-agent experiments were retained as exploratory diagnostics rather than a central contribution. In the updated appendix table, the Qwen2.5-VL-7B-Instruct-AWQ reviewer configuration underperforms its non-reviewer counterpart: pass rate drops from 47.18% to 40.94% on single-table and from 22.21% to 15.87% on multi-table. This supports the interpretation that downstream visual review cannot compensate for upstream data-grounding failures.

Recent ablation runs in the project results directory (for example, disabling Processor or Composer-template components) reinforce this conclusion: weakening schema filtering or structured planning typically reduces end-to-end pass rate, especially for multi-table queries. In practical terms, the highest-return improvements come from strengthening upstream reasoning and data integration before adding additional visual critique loops.

5.4. Discussion

Taken together, the results support three main conclusions.

1. **Local open-source models are practically competitive for NL2Vis when model choice is optimized.** Coder-specialized 14B–32B/30B models provide substantial gains over small general instruction baselines, particularly on multi-table tasks.

2. **The dominant technical bottleneck is semantic data grounding.** Invalid rates can be reduced significantly, but illegal rates remain high enough to cap

overall pass rate in difficult settings.

3. **Visual review is not the first-order lever in this setup.** Reviewer-agent experiments do not improve final pass outcomes when upstream legality checks fail.

These findings support a local-first deployment strategy for privacy-sensitive settings. At the same time, they make clear that further gains will likely come from better relational reasoning and query-to-schema alignment rather than additional downstream visual polishing.

6. Conclusion

This thesis examined how far a locally deployed, `nvAgent`-style multi-agent NL2Vis pipeline can be pushed with open-source models. The central result is clear: local deployment is viable for meaningful database-grounded performance when model specialization and scale are chosen carefully.

The research questions from Chapter 4 can now be answered directly:

1. **RQ1 (local vs. proprietary proximity):** local coder models substantially narrow the gap to reported proprietary baselines, with best observed performance at 75.40% (single-table) and 70.32% (multi-table) pass rate on updated `VisEval`.

2. **RQ2 (dominant failure category):** as model capability improves, invalid failures decrease strongly, but illegal failures remain the principal constraint, especially in multi-table settings.

3. **RQ3 (performance-efficiency trade-offs):** the largest gains come from moving from small general models to coder-specialized models and then to larger coder models; beyond that, improvements become more incremental and must be weighed against local compute budget.

Accordingly, the main contribution is not a single architectural trick, but a calibrated empirical map of what currently works in a privacy-preserving local NL2Vis stack: which model families are worth deploying, where failures concentrate, and which optimization direction is most likely to improve pass rate.

6.1. Limitations

Despite the encouraging results, several limitations should be acknowledged.

1. **Single-run determinism over repeated trials:** most reported results are deterministic single-run evaluations, which improves reproducibility but limits statistical uncertainty estimation.

2. **Infrastructure constraints:** experiments were conducted under bounded local/cluster hardware resources; this constrains broader sweeps over larger models and more repetitions. More specifically: H100 slice (40 GB MIG), 16 CPUs, 64 GB RAM, which limits the largest model and quantization configurations that can be tested.

3. **Readability pipeline dependencies:** readability and layout assessment depend on additional components (vision judge and browser tooling), which introduces cost and operational fragility relative to the core pass-rate evaluation.

6.2. Future Work

Future work should prioritize improvements in the same order as the observed bottlenecks:

1. **Data-grounding-first optimization:** improve schema filtering, join reliability, and query-to-VQL consistency checks in Processor/Composer stages.
2. **Targeted ablation expansion:** complete systematic ablations across model scales to isolate which agent components contribute most under local constraints.
3. **Interactive correction loops:** extend the system to multi-turn human-in-the-loop interaction with memory, then evaluate whether iterative user feedback improves legality outcomes.
4. **Benchmark extension:** augment current one-shot evaluation with controlled iterative scenarios, so correction ability can be measured directly rather than inferred.

In summary, this thesis shows that local NL2Vis is already strong enough to be relevant for many privacy-sensitive use cases. The next stage of progress will depend primarily on improving relational reasoning and data grounding, rather than adding more downstream visual critique.

Bibliography

Chen, N., Zhang, Y., Xu, J., Ren, K., & Yang, Y. (2025). *VisEval: A Benchmark for Data Visualization in the Era of Large Language Models. IEEE Transactions on Visualization and Computer Graphics*, 31(1), 1301–1311.

Dibia, V. (2023). *LIDA: A Tool for Automatic Generation of Grammar-Agnostic Visualizations and Infographics using Large Language Models.* In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics: System Demonstrations* (pp. 113–126). Association for Computational Linguistics.

Goswami, K., Mathur, P., Rossi, R., & Dernoncourt, F. (2025). *PlotGen: Multi-Agent LLM-based Scientific Data Visualization via Multimodal Feedback.* arXiv preprint arXiv:2502.00988. [1, 4, 18]

Huynh, N., & Lin, B. (2025). *Large Language Models for Code Generation: A Comprehensive Survey of Challenges, Techniques, Evaluation, and Applications.* arXiv preprint arXiv:2503.01245. [19]

Ouyang, G., Chen, J., Nie, Z., Gui, Y., Wan, Y., Zhang, H., & Chen, D. (2025). *nvAgent: Automated Data Visualization from Natural Language via Collaborative Agent Workflow.* In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers).* [3, 20]

Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J. E., Zhang, H., & Stoica, I. (2023). *Efficient Memory Management for Large Language Model Serving with PagedAttention.* In *Proceedings of the 29th Symposium on Operating Systems Principles* (SOSP '23).

Li, X., Wang, S., Zeng, S., Wu, Y., & Yang, Y. (2024). *A Survey on LLM-Based Multi-Agent Systems: Workflow, Infrastructure, and Challenges. Vicinagearth*, 1(1), 9.

Li, S., Chen, X., Song, Y., Song, Y., Zhang, C. J., Hao, F., & Chen, L. (2025). *Prompt4Vis: Prompting Large Language Models with Example Mining for Tabular Data Visualization. The VLDB Journal*, 34(4), article 38.

Lin, J., Tang, J., Tang, H., Yang, S., Chen, W.-M., Wang, W.-C., Xiao, G., Dang, X., Gan, C., & Han, S. (2024). *AWQ: Activation-Aware Weight Quantization for LLM Compression and Acceleration.* In *Proceedings of Machine Learning and Systems* (MLSys 2024). *Best Paper Award.*

Luo, Y., Tang, N., Li, G., Chai, C., Li, W., & Qin, X. (2021). *Synthesizing Natural Language to Visualization (NL2VIS) Benchmarks from NL2SQL Benchmarks.* In *Proceedings of the 2021 International Conference on Management of Data* (SIGMOD '21) (pp. 1235–1247).

Luo, Y., Tang, N., Li, G., Tang, J., Chai, C., & Qin, X. (2022). *Natural Language to Visualization by Neural Machine Translation. IEEE Transactions on Visualization and Computer Graphics*, 28(1), 217–226.

Maddigan, P., & Susnjak, T. (2023). *Chat2VIS: Generating Data Visualizations via Natural Language Using ChatGPT, Codex and GPT-3 Large Language Models. IEEE Access*, 11, 45181–45193.

Nguyen, H., He, X., Reeson, A., Paris, C., Poon, J., & Kummerfeld, J. K. (2024). *Do Text-to-Vis Benchmarks Test Real Use of Visualisations?.* arXiv preprint arXiv:2407.19726.

Khan, S. R., Chandak, V., & Mukherjea, S. (2025). *Evaluating LLMs for Visualization Generation and Understanding. Discover Data*, 3(1), 15.

Shi, S., Ren, T., Zhu, G., Feng, G., & Hu, J. (2025). *Closing the Feedback Loop in Text2Vis: Refining Visualization with Vision-Language Models.* In *Proceedings of the 33rd ACM International Conference on Multimedia* (MM '25) (pp. 9053–9061).

Voigt, H., Lawonn, K., & Zarrieß, S. (2024). *Plots Made Quickly: An Efficient Approach for Generating Visualizations from Natural Language Queries.* In *Proceedings of the 2024 Joint International Conference on Computational Linguistics, Language Resources and Evaluation* (LREC-COLING 2024) (pp. 12787–12793).

Wu, Y., Wan, Y., Zhang, H., Sui, Y., Wei, W., Zhao, W., Xu, G., & Jin, H. (2024). *Automated Data Visualization from Natural Language via Large Language Models: An Exploratory Study. Proceedings of the ACM on Management of Data*, 2(3), article 115.

Yang, Z., Zhou, Z., Wang, S., Cong, X., Han, X., Yan, Y., Liu, Z., Tan, Z., Liu, P., Yu, D., Liu, Z., Shi, X., & Sun, M. (2024). *MatPlotAgent: Method and Evaluation for LLM-Based Agentic Scientific Data Visualization.* In *Findings of the Association for Computational Linguistics: ACL 2024.* [7, 9, 21]

Zhang, R., & Elhamod, M. (2025). *Data-to-Dashboard: Multi-Agent LLM Framework for Insightful Visualization in Enterprise Analytics.* arXiv preprint arXiv:2505.23695. [11, 12]

Yu, T., Zhang, R., Yang, K., Yasunaga, M., Wang, D., Li, Z., Ma, J., Li, I., Yao, Q., Roman, S., Zhang, Z., & Radev, D. (2018). *Spider: A Large-Scale Human-Labeled Dataset for Complex and Cross-Domain Semantic Parsing and Text-to-SQL.* In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing* (pp. 3911–3921).

Zhu, X., Li, Q., Cui, L., & Liu, Y. (2024). *Large Language Model Enhanced Text-to-SQL Generation: A Survey.* arXiv preprint arXiv:2410.06011.

A. Additional Experimental Results

This appendix contains supplementary figures and tables from our experimental runs, including detailed performance breakdowns. GitHub link: https://github.com/DontSlipOnDirt/NL2Vis.

Model	Single-Table					Multi-Table				
	Invalid(↓)	Illegal(↓)	Pass(↑)	Read.(↑)	Qual.(↑)	Invalid(↓)	Illegal(↓)	Pass(↑)	Read.(↑)	Qual.(↑)
NVAGENT Paper										
Qwen2.5-7B-Instruct	11.71%	28.40%	59.87%	3.61	2.17	25.14%	33.83%	41.02%	3.61	1.48
GPT-3.5-turbo	2.98%	20.93%	76.08%	3.58	2.72	7.18%	28.51%	64.29%	3.61	2.32
GPT-4o-mini	1.97%	22.86%	75.16%	3.67	2.77	8.15%	25.99%	65.85%	3.66	2.42
GPT-4o	0.72%	13.63%	85.63%	3.66	3.13	1.34%	17.57%	81.07%	3.61	2.93
Locally with vLLM										
Qwen2.5-VL-7B-Instruct-AWQ	8.66%	44.15%	47.18%	–	–	29.80%	47.99%	22.21%	–	–
Qwen2.5-7B-Instruct	6.98%	41.31%	51.72%	–	–	31.67%	42.31%	26.02%	–	–
Qwen2.5-Coder-7B-Instruct	5.59%	38.59%	55.82%	–	–	8.15%	48.19%	43.65%	–	–
Qwen2.5-Coder-14B-Instruct	3.08%	34.57%	62.35%	3.27	2.02	7.97%	40.42%	51.60%	3.25	1.66
Qwen2.5-Coder-32B-Instruct	5.7%	30.46%	63.84%	3.36	2.13	9.15%	40.17%	50.67%	3.29	1.66
Qwen3-Coder-30B-A3B-Instruct	4.16%	32.48%	63.37%	3.34	2.10	8.06%	39.13%	52.81%	3.30	1.74
With reviewer agent										
Qwen2.5-VL-7B-Instruct-AWQ	18.16%	40.90%	40.94%	–	–	48.05%	36.08%	15.87%	–	–
After VisEval Update										
Qwen2.5-7B-Instruct	8.48%	28.88%	62.64%	3.49	2.20	29.78%	32.98%	37.25%	3.51	1.30
Qwen2.5-Coder-7B-Instruct-AWQ	7.55%	27.37%	65.08%	–	–	13.01%	34.58%	52.41%	–	–
Qwen2.5-Coder-14B-Instruct-AWQ	4.77%	21.68%	73.54%	3.37	2.49	9.81%	26.73%	63.46%	3.37	2.13
Qwen2.5-Coder-32B-Instruct-AWQ	5.67%	21.84%	72.50%	3.57	2.59	10.27%	26.01%	63.72%	3.70	2.37
Qwen3-Coder-30B-A3B-Instruct-FP8	4.79%	20.06%	75.15%	–	–	7.53%	21.66%	70.81%	–	–
w/o Processor										
Qwen2.5-Coder-14B-Instruct-AWQ	4.73%	21.71%	73.56%	–	–	9.45%	26.87%	63.68%	–	–
Qwen2.5-Coder-32B-Instruct-AWQ	7.05%	17.73%	75.21%	–	–	9.85%	26.64%	63.50%	–	–
Qwen3-Coder-30B-A3B-Instruct-FP8	4.60%	19.41%	75.99%	–	–	5.36%	24.22%	70.42%	–	–

Table A.1.: Detailed single-table and multi-table metrics across reported baselines and local runs, including the post-update `VisEval` results used in the main chapter analysis.

A.1. Inference Performance Analysis

A.2. Ablation Study Results

A.3. Check Failure Analysis

A.4. Quantization Impact Comparison

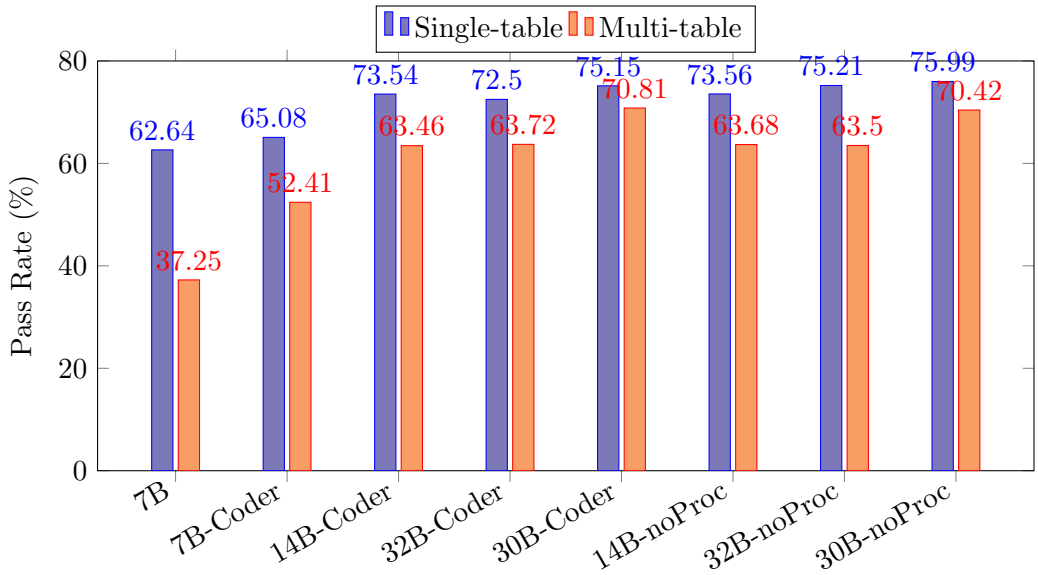

Figure A.1.: Pass-rate overview from Table A.1, comparing updated full-pipeline runs and no-Processor ablations.

Model	**Avg. Inference Time (s)**	**Total Inference Time (s)**	**Avg. Iterations per Query**	**Pass Rate**
		With Processor (Full Pipeline)		
Qwen3-Coder-30B-A3B-FP8	4.25	8.62	2.19	73.44%
		Without Processor (Ablation)		
Qwen3-Coder-30B-A3B-FP8	2.00	3.74	2.19	73.80%

Table A.2.: Inference time and iteration metrics with and without the Processor agent. The Processor adds minimal latency while improving data integration quality, though pass rate improvements are modest in these configurations.

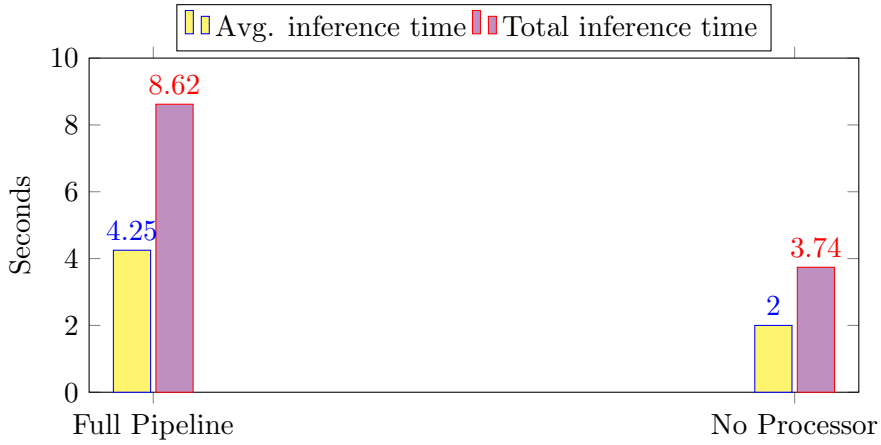

Figure A.2.: Inference-time comparison from Table A.2 for Qwen3-Coder-30B-A3B-FP8 with and without Processor.

Configuration	**Single-Table**			**Multi-Table**		
	Invalid	Illegal	Pass	Invalid	Illegal	Pass
Qwen2.5-Coder-14B-Instruct-AWQ						
Full Pipeline	4.77%	21.68%	**73.54%**	9.81%	26.73%	**63.46%**
No Processor	4.73%	21.71%	73.56%	9.45%	26.87%	63.68%
No Composer Template	8.07%	88.72%	3.21%	29.10%	69.79%	1.11%
No Proc. + No Comp. Tmpl.	8.56%	87.88%	3.56%	29.82%	68.83%	1.35%
Qwen2.5-Coder-32B-Instruct-AWQ						
Full Pipeline	5.67%	21.84%	**72.50%**	10.27%	26.01%	**63.72%**
No Processor	7.05%	17.73%	75.21%	9.85%	26.64%	63.50%
No Composer Template	11.36%	83.03%	5.60%	13.57%	85.43%	1.00%
Qwen3-Coder-30B-A3B-Instruct-FP8						
Full Pipeline	4.79%	20.06%	75.15%	7.53%	21.66%	**70.81%**
No Processor	4.60%	19.41%	**75.99%**	5.36%	24.22%	70.42%

Table A.3.: Ablation study results showing the impact of removing key pipeline components. The Composer Template is critical—removing it causes catastrophic failure (pass rate drops below 5%). The Processor shows minimal impact on final pass rates, though it aids in schema filtering for complex databases.

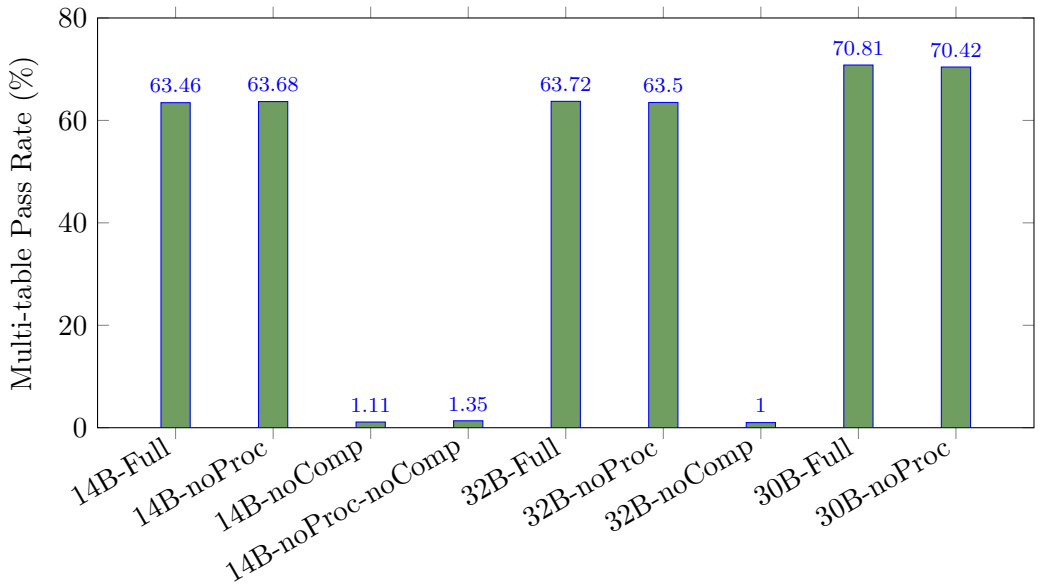

Figure A.3.: Multi-table ablation comparison from Table A.3. Removing the Composer template causes near-collapse across model sizes.

Model	Code Exec.	Data Check	Chart Type	Order Check	Scale & Ticks	Pass Rate
Qwen2.5-7B	15.44%	**21.24%**	2.80%	9.25%	26.00%	52.66%
Qwen2.5-Coder-7B-AWQ	9.05%	**20.76%**	3.33%	9.45%	–	60.10%
Qwen2.5-Coder-14B-AWQ	6.58%	**16.63%**	1.77%	6.87%	37.95%	69.58%
Qwen2.5-Coder-32B-AWQ	6.80%	**16.33%**	1.51%	6.49%	27.58%	69.04%
Qwen3-Coder-30B-A3B-FP8	5.78%	**13.33%**	1.82%	7.31%	–	73.44%
Ablation: No Composer Template						
Qwen2.5-Coder-14B-AWQ	7.26%	**80.92%**	**79.39%**	0.36%	–	2.39%

Table A.4.: Frequency of check failures across evaluation metrics. **Data Check** is consistently the dominant failure mode, accounting for 15–27% of failures in functional configurations. Without the Composer Template, data and chart type failures exceed 79%, confirming that structured intermediate representation is essential for correctness.

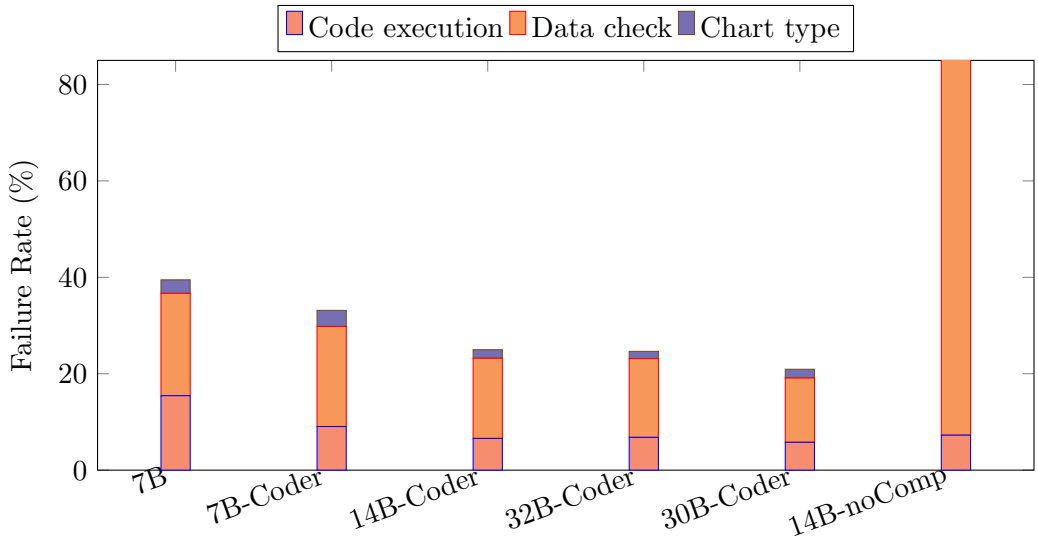

Figure A.4.: Stacked failure composition from Table A.4. Data and chart-type checks dominate in the no-composer ablation.

Model Variant	**Single-Table**			**Multi-Table**		
	Invalid	Illegal	Pass	Invalid	Illegal	Pass
7B Models						
Qwen2.5-7B-Instruct (FP16)	8.48%	28.88%	62.64%	29.78%	32.98%	37.25%
Qwen2.5-Coder-7B-AWQ (INT4)	7.55%	27.37%	65.08%	13.01%	34.58%	52.41%
Δ	-0.93%	-1.51%	+2.44%	-16.77%	+1.60%	+15.16%
14B Models						
Qwen2.5-Coder-14B (FP16)	3.08%	34.57%	62.35%	7.97%	40.42%	51.60%
Qwen2.5-Coder-14B-AWQ (INT4)	4.77%	21.68%	73.54%	9.81%	26.73%	63.46%
Δ	+1.69%	-12.89%	+11.19%	+1.84%	-13.69%	+11.86%
32B Models						
Qwen2.5-Coder-32B-AWQ (INT4)	5.67%	21.84%	72.50%	10.27%	26.01%	63.72%
Δ	-0.03%	-8.62%	+8.66%	+1.12%	-14.16%	+13.05%

Table A.5.: Quantization impact on model performance. AWQ (INT4) quantization improves both pass rates and illegal rates across all model sizes, likely due to updated `VisEval` benchmark and refined prompting in quantized runs. The 7B coder-specialized quantized model substantially outperforms the FP16 general instruction model, confirming that code specialization matters more than quantization precision loss.

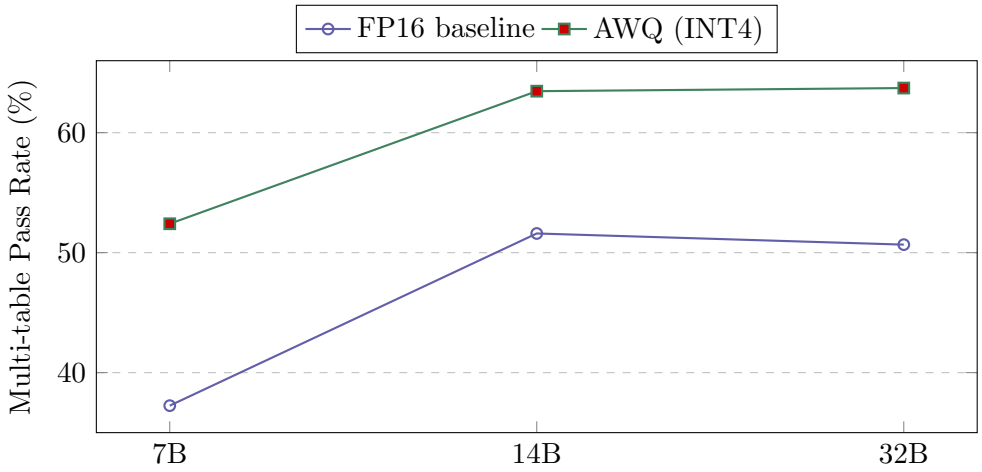

Figure A.5.: Quantization trend from Table A.5: AWQ consistently improves multi-table pass rate across model scales.

B. Ehrenwörtliche Erklärung

Ich versichere, dass ich die beiliegende Bachelor-, Master-, Seminar-, oder Projektarbeit ohne Hilfe Dritter und ohne Benutzung anderer als der angegebenen Quellen und in der untenstehenden Tabelle angegebenen Hilfsmittel angefertigt und die den benutzten Quellen wörtlich oder inhaltlich entnommenen Stellen als solche kenntlich gemacht habe. Diese Arbeit hat in gleicher oder ähnlicher Form noch keiner Prüfungsbehörde vorgelegen. Ich bin mir bewusst, dass eine falsche Erklärung rechtliche Folgen haben wird.

Declaration of Used AI Tools

Tool	Purpose	Where?	Useful?
ChatGPT	Initial drafting, literature review	Chapters 1-2	++
Claude Sonnet 3.5	Code Generation (Initial Agent Framework)	Chapter 3	+
Llama 3 8B	Local LLM for Processor/Composer Agents	Chapters 4-5	++
Code Llama 7B	Local LLM for Synthesizer Agent	Chapters 4-5	++
LLaVA 1.5 7B	Local VLM for Multimodal Critic Agent	Chapters 4-5	+

Unterschrift:
Jacob Chen
Mannheim, den 4. Mai 2026